

Please recognize the four images: text.tif, test-spotshade.tif, text-broken.tif, and text-sinshade.tif. Please describe your method, procedures, final text file and print out the source code?

1. text-broken.tif

```
import cv2
import numpy as np
import pytesseract

def preprocess_text_broken(img_gray):
    """
    對 text-broken.tif 做前處理：
    1. Otsu 二值化
    2. 統一成白字黑底
    3. 使用 5x5 形態學 closing 接合破碎文字
    4. 去除太小的雜點
    回傳處理後的「白字黑底」二值影像
    """
    # 1. Otsu 二值化
    _, bw = cv2.threshold(img_gray, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # 2. 統一成白字黑底
    num_white = np.sum(bw == 255)
    num_black = bw.size - num_white
    if num_white > num_black:
        bw = 255 - bw

    # 3. 形態學 closing 接合破碎文字
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    closed = cv2.morphologyEx(bw, cv2.MORPH_CLOSE, kernel, iterations=1)

    # 4. 去除太小的雜點
    num_labels, labels, stats, _ =
cv2.connectedComponentsWithStats(closed)
    min_area = 20
    clean = np.zeros_like(closed)
    for i in range(1, num_labels):
        area = stats[i, cv2.CC_STAT_AREA]
```

```

        if area >= min_area:
            clean[labels == i] = 255

    return clean    # 白字黑底

def ocr_text_broken(bin_img):
    """
    對前處理後的二值影像做 OCR：
    1. 轉成黑字白底
    2. 放大 2 倍
    3. 呼叫 Tesseract
    """
    # 1. 白字黑底 → 黑字白底
    img_ocr = 255 - bin_img

    # 2. 放大
    img_ocr = cv2.resize(
        img_ocr, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_LINEAR
    )

    # 3. Tesseract
    config = "--psm 3"
    text = pytesseract.image_to_string(img_ocr, config=config)
    return text, img_ocr

def main():
    img_gray = cv2.imread("text-broken.tif", cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        raise FileNotFoundError("找不到 text-broken.tif, 請確認檔案是否在同一資料夾。")

    # 前處理與儲存影像（報告用）
    bin_clean = preprocess_text_broken(img_gray)
    cv2.imwrite("text-broken_processed.png", bin_clean)

    # OCR
    text, img_ocr = ocr_text_broken(bin_clean)
    cv2.imwrite("text-broken_ocr_input.png", img_ocr)

    # ===== 整理文字：避免多出原圖沒有的空白行 =====

```

```

# 1. 去掉 form feed 控制字元
text = text.replace('\x0c', '')

# 2. 切成一行一行
lines = text.splitlines()

# 3. 去掉開頭/結尾的空白行
while lines and lines[0].strip() == '':
    lines.pop(0)
while lines and lines[-1].strip() == '':
    lines.pop()

# 4. 移除「前後都有文字、自己卻是空白」的孤立空白行
cleaned_lines = []
for i, ln in enumerate(lines):
    if ln.strip() != '':
        cleaned_lines.append(ln)
    else:
        prev_has_text = (i > 0 and lines[i - 1].strip() != '')
        next_has_text = (i < len(lines) - 1 and lines[i + 1].strip()
!= '')

        # 若前後都有文字，就判定為多出來的空行 → 丟掉
        if prev_has_text and next_has_text:
            continue

        # 否則保留（例如連續多行空白，用來分段的情況）
        cleaned_lines.append(ln)

# 5. 組回字串，最後補一個換行
text = '\n'.join(cleaned_lines) + '\n'

# =====

with open("text-broken.txt", "w", encoding="utf-8") as f:
    f.write(text)

print("辨識完成，結果已寫入 text-broken.txt")
print("==== OCR 輸出預覽 =====")
print(text)

if __name__ == "__main__":
    main()

```

接合後的影像：

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

辨識文字：

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

方法步驟：

1. 讀入影像 (灰階)
用 OpenCV 讀 `text-broken.tif` 成灰階影像。
2. Otsu 二值化
使用 Otsu 自動閾值, 把影像變成黑白:
 - 背景一種顏色
 - 文字另一種顏色
3. 統一成「白字黑底」來做形態學處理
 - 根據黑白像素數判斷「背景是白」還是「背景是黑」,
 - 如果背景是白、文字是黑, 就先反相, 變成 白字黑底, 方便後面做膨脹/侵蝕。
4. 形態學 closing 接合破碎文字
 - 使用 3×3 橢圓形結構元素做 closing (膨脹再侵蝕):
 - closing 可以填補筆畫中的小縫隙,
 - 把原本斷掉的文字片段接在一起, 變成完整的字。
5. 移除面積太小的雜點
 - 用連通元件標記 (connected components), 把每個 blob 的面積算出來,
 - 面積太小的視為雜訊, 直接去掉, 只留下真正的文字。
6. 準備給 OCR: 改成黑字白底並放大
 - Tesseract 通常較習慣黑字白底, 所以最後再反相一次, 變成黑字白底。
 - 為了讓筆畫更清楚, 將影像等比例放大 2 倍, 有助於 OCR 辨識。
7. 使用 Tesseract OCR 辨識文字
 - 用 `pytesseract.image_to_string` 對處理完的影像做文字辨識。
 - 這裡只用傳統 OCR 引擎, 不用 LLM, 不看圖手動輸入。
8. 把辨識結果寫入 `text-broken.txt`
 - 將文字結果存成 `text-broken.txt` (作業規定的檔名)。

2. text.tif

```
import cv2
import numpy as np
import pytesseract

def preprocess_text_clear_border(img_gray):
    """
    對 text.tif 做前處理：
    1. Otsu 二值化
    2. 統一成白字黑底
    3. 利用連通元件標記，刪除所有接觸影像邊緣的字元
    回傳處理後的「白字黑底」二值影像
    """
    # 1. Otsu 二值化
    _, bw = cv2.threshold(img_gray, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # 2. 統一成白字黑底
    num_white = np.sum(bw == 255)
    num_black = bw.size - num_white
    # 若白色比較多，通常是白底黑字 → 反相成白字黑底
    if num_white > num_black:
        bw = 255 - bw

    h, w = bw.shape

    # 3. 連通元件標記
    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(bw)

    # 準備輸出影像，先全部設為黑
    result = np.zeros_like(bw)

    # 0 是背景，從 1 開始走訪每個元件
    for i in range(1, num_labels):
        x = stats[i, cv2.CC_STAT_LEFT]
        y = stats[i, cv2.CC_STAT_TOP]
        ww = stats[i, cv2.CC_STAT_WIDTH]
        hh = stats[i, cv2.CC_STAT_HEIGHT]

        # 算出此元件 bounding box 的四邊座標
        left = x
        top = y
```

```

    right = x + ww - 1
    bottom = y + hh - 1

    # 檢查是否碰觸到影像邊界
    touches_border = (
        left == 0 or
        top == 0 or
        right == w - 1 or
        bottom == h - 1
    )

    if touches_border:
        # 接觸邊界 → 整個字元刪除 (不寫進 result)
        continue

    # 不接觸邊界 → 保留在結果影像中
    result[labels == i] = 255

return result # 白字黑底, 邊緣字母已被清除

def ocr_text(bin_img):
    """
    對前處理後的二值影像做 OCR:
    1. 轉成黑字白底
    2. 放大 2 倍
    3. 呼叫 Tesseract
    """
    # 1. 白字黑底 → 黑字白底
    img_ocr = 255 - bin_img

    # 2. 放大
    img_ocr = cv2.resize(
        img_ocr, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_LINEAR
    )

    # 3. 呼叫 Tesseract
    # --psm 3: 把整張視為一個文字區塊
    config = "--psm 3"
    text = pytesseract.image_to_string(img_ocr, config=config)
    return text, img_ocr

```

```

def clean_ocr_text(text: str) -> str:
    """
    整理 Tesseract 的輸出：
    - 去掉 form feed (\x0c)
    - 去掉開頭/結尾空白行
    - 移除「前後都有文字、自己卻是空白」的孤立空白行
    """
    # 去掉控制字元
    text = text.replace('\x0c', '')

    # 切成一行一行
    lines = text.splitlines()

    # 去掉開頭/結尾空白行
    while lines and lines[0].strip() == '':
        lines.pop(0)
    while lines and lines[-1].strip() == '':
        lines.pop()

    # 移除中間「多出來」的孤立空白行
    cleaned_lines = []
    for i, ln in enumerate(lines):
        if ln.strip() != '':
            cleaned_lines.append(ln)
        else:
            prev_has_text = (i > 0 and lines[i - 1].strip() != '')
            next_has_text = (i < len(lines) - 1 and lines[i + 1].strip()
!= '')
            # 前後都有文字 → 判定為多出來的空行，直接丟掉
            if prev_has_text and next_has_text:
                continue
            cleaned_lines.append(ln)

    # 再組回字串，並在最後補一個換行
    return '\n'.join(cleaned_lines) + '\n'

def main():
    # 讀入 text.tif (灰階)
    img_gray = cv2.imread("text.tif", cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        raise FileNotFoundError("找不到 text.tif，請確認檔案是否在同一資料夾。")
    ")

```

```

# 前處理：刪除碰到邊緣的字母
bin_clean = preprocess_text_clear_border(img_gray)

# 儲存清除後的影像（白字黑底），可放到報告中
cv2.imwrite("text_processed.png", bin_clean)

# OCR 辨識
raw_text, img_ocr = ocr_text(bin_clean)

# 儲存送進 Tesseract 的影像（黑字白底、放大後）
cv2.imwrite("text_ocr_input.png", img_ocr)

# 整理文字：避免多出原圖沒有的空白行
final_text = clean_ocr_text(raw_text)

# 寫入 text.txt（作業要求）
with open("text.txt", "w", encoding="utf-8") as f:
    f.write(final_text)

print("text.tif 辨識完成，結果已寫入 text.txt")
print("==== OCR 輸出預覽（前幾行）====")
print('\n'.join(final_text.splitlines()[:10]))

if __name__ == "__main__":
    main()

```

清除後的影像：

ponents or broken connection paths. There is no position past the level of detail required to identify those

Segmentation of nontrivial images is one of the most important steps in image processing. Segmentation accuracy determines the effectiveness of computerized analysis procedures. For this reason, great care must be taken to improve the probability of rugged segmentation. In applications such as industrial inspection applications, at least some degree of ruggedness in the environment is possible at times. The experienced designer invariably pays considerable attention to such

辨識文字：

ponents or broken connection paths. There is no portion past the level of detail required to identify those Segmentation of nontrivial images is one of the most difficult parts of image processing. Segmentation accuracy determines the effectiveness of computerized analysis procedures. For this reason, segmentation should be taken to improve the probability of rugged segmentation in applications, at least some of which are possible at times. The experienced designer invariably pays considerable attention to such

方法步驟：

1. 讀入灰階影像
用 OpenCV 把 `text.tif` 讀成灰階矩陣。
2. Otsu 二值化，得到文字區域
使用 Otsu 自動閾值把影像變成黑白影像。
3. 統一成「白字黑底」
 - 統計白、黑像素數量，
 - 若白色比較多，代表背景是白、文字是黑 → 反相，讓「文字 = 白，背景 = 黑」，方便做連通元件。
4. 連通元件標記 + 刪除碰到邊緣的元件
 - 對白色區域做 connected components labeling。
 - 對每個元件拿到它的 bounding box (左上角 x, y 與寬高 w, h)。
 - 如果此元件的 bounding box 碰到影像的上、下、左或右邊界，代表這個字貼在邊界上 → 直接整個丟掉。
 - 其餘元件保留，形成「已經把邊緣文字清掉」的二值影像。
5. (可選) 移除面積超小的雜點
如有需要，可以用元件面積門檻過濾掉很小的噪點。
6. 準備給 OCR
 - 反相成「黑字白底」(Tesseract 比較習慣)。
 - 將影像放大 2 倍，提高辨識品質。
7. Tesseract OCR 辨識文本

- 使用 `pytesseract.image_to_string` 辨識文字。
- 不用任何大型語言模型, 也不手動看圖輸入, 符合作業要求。

8. 整理文字並寫入 `text.txt`

- 去掉 Tesseract 會加的 `\x0c` 控制字元。
- 移除多餘的空白行 (例如開頭、結尾或單獨多出來的一行空白)。
- 將結果寫成 `text.txt` (繳交要求的檔名)。

3. text-sineshade.tif

```
import cv2
import numpy as np
import pytesseract

def preprocess_text_sineshade(img_gray, ksize=101):
    """
    對 text-sineshade.tif 做前處理：
    1. 以大尺寸高斯模糊估計背景 (sine shade)
    2. 以除法方式做光照補償，減少條紋陰影
    3. Otsu 二值化，得到黑白文字影像
    回傳：
        corrected : 補償後的灰階影像
        bw        : 二值影像
    """
    # 轉成 float，避免除法時溢位，並加 1 避免除以 0
    img_f = img_gray.astype(np.float32) + 1.0

    # 大 kernel 高斯模糊估計背景 (只保留緩慢變化的 sine shade)
    bg = cv2.GaussianBlur(img_f, (ksize, ksize), 0) + 1.0

    # 光照補償：除以背景，讓背景變得較平均
    corrected = (img_f / bg) * 128.0
    corrected = np.clip(corrected, 0, 255).astype(np.uint8)

    # Otsu 二值化
    _, bw = cv2.threshold(corrected, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    return corrected, bw

def ocr_from_binary(bw):
    """
    使用二值影像做 OCR：
    1. 統一成黑字白底 (Tesseract 比較習慣)
    2. 放大 2 倍
    3. 呼叫 Tesseract
    """
    # 判斷目前是白底黑字還是黑底白字
    num_white = np.sum(bw == 255)
    num_black = bw.size - num_white
```

```

# 我們要「黑字白底」給 OCR
if num_white >= num_black:
    img_ocr = bw.copy()          # 背景是白, 字是黑 → OK
else:
    img_ocr = 255 - bw          # 反相

# 放大 2 倍
img_ocr = cv2.resize(
    img_ocr, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_LINEAR
)

# Tesseract 參數: --psm 6 適合一般文字段落
config = "--psm 6"
text = pytesseract.image_to_string(img_ocr, config=config)

return text, img_ocr

def clean_ocr_text(text: str) -> str:
    """
    整理 Tesseract 的輸出:
    - 去掉 form feed (\x0c)
    - 去掉開頭/結尾空白行
    - 移除「前後都有文字、自己卻是空白」的孤立空白行
    """
    # 去掉控制字元
    text = text.replace('\x0c', '')

    # 切成行
    lines = text.splitlines()

    # 去掉開頭/結尾空白行
    while lines and lines[0].strip() == '':
        lines.pop(0)
    while lines and lines[-1].strip() == '':
        lines.pop()

    # 移除中間的孤立空白行
    cleaned_lines = []
    for i, ln in enumerate(lines):
        if ln.strip() != '':
            cleaned_lines.append(ln)

```

```

        else:
            prev_has_text = (i > 0 and lines[i - 1].strip() != '')
            next_has_text = (i < len(lines) - 1 and lines[i + 1].strip()
!= '')

            # 前後都有文字 → 視為多出來的空白行，丟掉
            if prev_has_text and next_has_text:
                continue
            cleaned_lines.append(ln)

# 組回字串，最後補一個換行
return '\n'.join(cleaned_lines) + '\n'

def main():
    # 讀入 text-sineshade.tif (灰階)
    img_gray = cv2.imread("text-sineshade.tif", cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        raise FileNotFoundError("找不到 text-sineshade.tif, 請確認檔案是否在
同一資料夾。")

    # 1. 去除 sine shade (背景補償 + 二值化)
    corrected, bw = preprocess_text_sineshade(img_gray, ksize=101)

    # 儲存「清除 sine shade 後」的影像 (報告可用)
    cv2.imwrite("text-sineshade_corrected.png", corrected)
    cv2.imwrite("text-sineshade_binary.png", bw)

    # 2. OCR 辨識
    raw_text, img_ocr = ocr_from_binary(bw)

    # 儲存餵給 Tesseract 的影像 (黑字白底、放大後)
    cv2.imwrite("text-sineshade_ocr_input.png", img_ocr)

    # 3. 整理文字，避免多出原圖沒有的空行
    final_text = clean_ocr_text(raw_text)

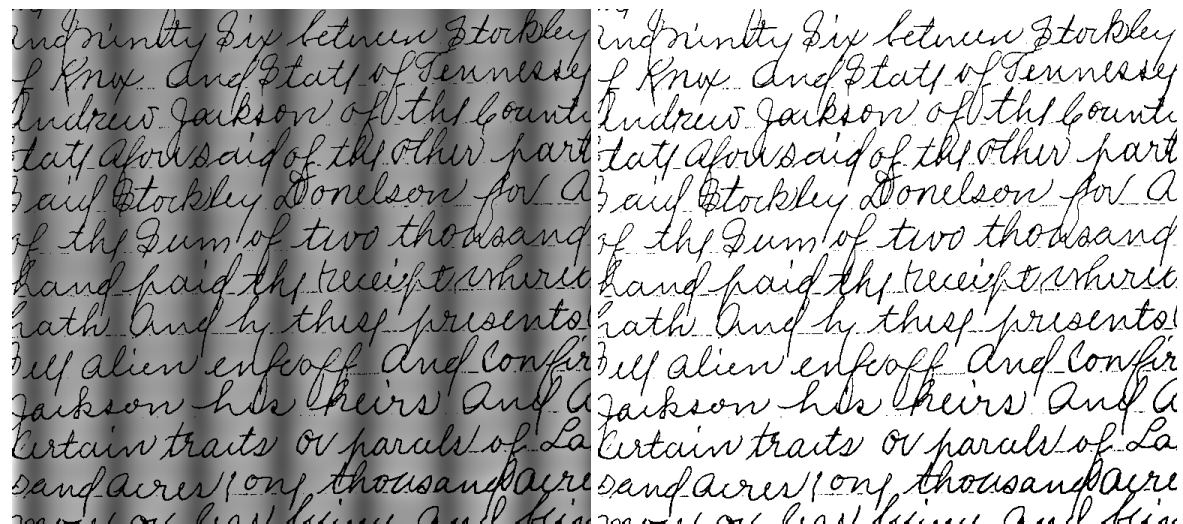
    # 4. 寫入文字檔 (作業規定檔名)
    with open("text-sineshade.txt", "w", encoding="utf-8") as f:
        f.write(final_text)

    print("text-sineshade.tif 辨識完成，結果已寫入 text-sineshade.txt")
    print("==== OCR 輸出預覽 ====")
    print('\n'.join(final_text.splitlines()[:10]))

```

```
if __name__ == "__main__":
    main()
```

清除後的影像：



* 先利用大尺寸高斯模糊估計並補償背景照明，再以 Otsu 閾值二值化。
可以看到 sine 已被清除，背景變為均勻白色，僅留下文字。

辨識文字：

iy IU ty Bip Aebtrw
[is tlie
ee eee

* 手寫書體字難以辨識

方法步驟：

1. 讀入灰階影像
用 OpenCV 讀取 `text-sinshade.tif`，得到灰階影像。
2. 背景估計(去除 sine shade 的核心)
 - sine shade 是一個「緩慢變化的亮度條紋」，變化頻率遠低於文字的細節。
 - 我們用一個很大的高斯濾波器(例如 101×101)去模糊整張圖，得到一張只保留緩慢變化亮度的「背景影像」 $B(x,y)$ 。
 - 原始影像 $I(x,y)$ 除以背景：

$$I_{corr} = I \times \frac{1}{B} \quad I_{corr} = \frac{I}{B}$$

可以把條紋大幅平坦化，讓背景更均勻，只留下文字的細節。

3. 二值化

- 對補償後影像 `IcorrI_{\text{corr}}` 使用 Otsu 閾值做二值化。
- 得到黑白影像，文字和背景對比變強，`sine shade` 消失。

4. OCR 辨識

- 將二值影像統一成「黑字白底」、放大 2 倍後，
- 使用 Tesseract OCR (`pytesseract`) 辨識文字內容。
- 全程只用傳統 OCR，不用大型語言模型，也沒有人工看圖打字。

5. 整理文字並寫入 `text-sinешade.txt`

- 去掉 Tesseract 附加的控制字元 `\x0c`。
- 切成一行一行，去掉開頭與結尾多餘空白行，
- 移除那種「前一行有字、下一行也有字，但中間多出一行空白」的孤立空白行，
避免比原圖多出奇怪的空白行。
- 結果寫入 `text-sinешade.txt`。

6. 輸出影像給報告用

- 輸出兩張圖：
 - i. `text-sinешade_corrected.png`: 陰影已被補償的灰階圖。
 - ii. `text-sinешade_binary.png`: 最後用來 OCR 的二值影像。
- 報告中可以放其中一張或兩張都放。

4. `text-spotshade.tif`

```

import cv2
import numpy as np
import pytesseract

def preprocess_text_spotshade(img_gray, ksize=101):
    """
    對 text-spotshade.tif 做前處理：
    1. 以大尺寸高斯模糊估計背景 (spot shade)
    2. 以除法方式做光照補償，減少中心亮、邊緣暗的現象
    3. Otsu 二值化得到黑白文字影像

    回傳：
        corrected : 補償後的灰階影像
        bw        : 二值影像
    """
    # 1. 轉成 float, 避免除法溢位, 加 1 避免除以 0
    img_f = img_gray.astype(np.float32) + 1.0

    # 2. 以大 kernel 高斯模糊估計平滑背景 (spot shade)
    bg = cv2.GaussianBlur(img_f, (ksize, ksize), 0) + 1.0

    # 3. 光照補償：原圖 / 背景 * 128 讓背景變得較平均
    corrected = (img_f / bg) * 128.0
    corrected = np.clip(corrected, 0, 255).astype(np.uint8)

    # 4. Otsu 二值化
    _, bw = cv2.threshold(corrected, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    return corrected, bw

def ocr_from_binary(bw):
    """
    使用二值影像做 OCR：
    1. 統一成黑字白底
    2. 放大 2 倍
    3. 呼叫 Tesseract
    """
    # 判斷目前是白底黑字還是黑底白字
    num_white = np.sum(bw == 255)
    num_black = bw.size - num_white

```

```

# 我們要「黑字白底」給 Tesseract
if num_white >= num_black:
    img_ocr = bw.copy()          # 背景白、字黑 → OK
else:
    img_ocr = 255 - bw          # 反相

# 放大 2 倍, 讓線條不要太細
img_ocr = cv2.resize(
    img_ocr, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_LINEAR
)

# Tesseract 設定: --psm 6 當作一般段落文字
config = "--psm 6"
text = pytesseract.image_to_string(img_ocr, config=config,
lang="eng")

return text, img_ocr

def clean_ocr_text(text: str) -> str:
    """
    整理 Tesseract 的輸出:
    - 去掉 form feed (\x0c)
    - 去掉開頭/結尾空白行
    - 移除「前後都有文字、自己卻是空白」的孤立空白行
    """
    # 去掉控制字元
    text = text.replace('\x0c', '')

    # 切成一行一行
    lines = text.splitlines()

    # 去掉開頭/結尾空白行
    while lines and lines[0].strip() == '':
        lines.pop(0)
    while lines and lines[-1].strip() == '':
        lines.pop()

    # 移除中間的孤立空白行
    cleaned_lines = []
    for i, ln in enumerate(lines):
        if ln.strip() != '':

```

```

        cleaned_lines.append(ln)
    else:
        prev_has_text = (i > 0 and lines[i - 1].strip() != '')
        next_has_text = (i < len(lines) - 1 and lines[i + 1].strip()
!= '')

        if prev_has_text and next_has_text:
            # 前後都有文字 → 視為多出來的空行，丟掉
            continue
        cleaned_lines.append(ln)

# 組回字串，最後補一個換行
return '\n'.join(cleaned_lines) + '\n'

def main():
    # 讀入 text-spotshade.tif (灰階)
    img_gray = cv2.imread("text-spotshade.tif", cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        raise FileNotFoundError("找不到 text-spotshade.tif，請確認檔案是否在
同一資料夾。")

    # 1. 去除 spot shade + Otsu 二值化
    corrected, bw = preprocess_text_spotshade(img_gray, ksize=101)

    # 這兩張影像可以放進報告中
    cv2.imwrite("text-spotshade_corrected.png", corrected)
    cv2.imwrite("text-spotshade_binary.png", bw)

    # 2. OCR 辨識
    raw_text, img_ocr = ocr_from_binary(bw)
    cv2.imwrite("text-spotshade_ocr_input.png", img_ocr)

    # 3. 整理文字
    final_text = clean_ocr_text(raw_text)

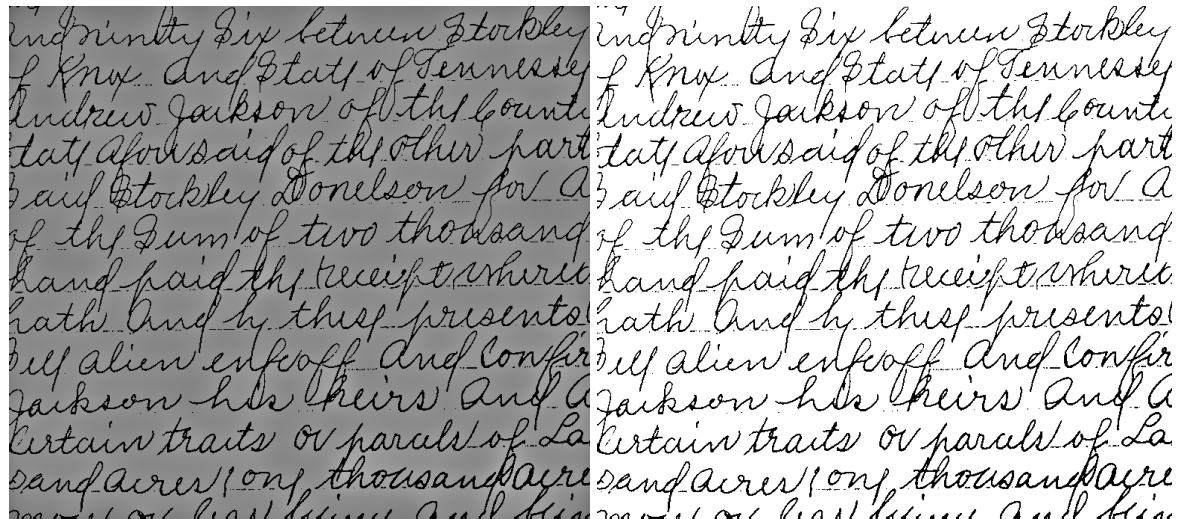
    # 4. 輸出文字檔 (作業規定)
    with open("text-spotshade.txt", "w", encoding="utf-8") as f:
        f.write(final_text)

    print("text-spotshade.tif 辨識完成，結果已寫入 text-spotshade.txt")
    print("==== OCR 輸出預覽 (前幾行) ====")
    print('\n'.join(final_text.splitlines()[:10]))

```

```
if __name__ == "__main__":
    main()
```

清除後的影像：



* 先利用大尺寸高斯模糊估計並補償背景照明，再以 Otsu 閾值二值化。
可以看到 spot shade 已被清除，背景變為均勻白色，僅留下文字。

辨識文字：

iy IU ty Bip Aebrtw
[is tlie
ee eee

* 手寫書體字難以辨識

方法步驟：

1. 讀入灰階影像
使用 OpenCV 讀取 `text-spotshade.tif` 成灰階影像。
2. 用大尺寸高斯模糊估計 spot shade 背景
 - `text-spotshade.tif` 中間比較亮、四周比較暗，是一種「spot / vignetting」式不均勻照明。
 - 對原圖做大 kernel (例如 101×101) 的高斯模糊，得到一張只保留緩慢亮度變化的背景影像 $B(x,y)$ 。
3. 光照補償 (去除 spot shade)
 - 將原影像 $I(x,y)$ 先轉成 float，做除法補償：

$$I_{corr} = I \times \frac{128}{B} \quad I_{corr} = B \times 128$$

- 這樣可把中心亮、邊緣暗的現象拉回比較均勻的背景，文字對比變清楚。

4. Otsu 二值化

- 對補償後影像 `lcorr_{\text{corr}}lcorr` 使用 Otsu 閾值做二值化，得到黑白影像。
- 此時 spot shade 已基本被消除，只剩黑色筆畫與白色背景。

5. OCR(Tesseract)

- 將二值影像統一成「黑字白底」，放大 2 倍後送進 Tesseract。
- 使用 `--psm 6`，視作一般文字段落進行辨識。
- 不使用大型語言模型、也不人工看圖輸入，符合作業規則。

6. 整理文字並寫入 `text-spotshade.txt`

- 去掉 Tesseract 會加的 `\x0c` 控制字元。
- 切行、移除開頭和結尾多餘空白行，以及「前後都有文字、卻多出一行空白」的孤立空行。
- 將乾淨的文字輸出到 `text-spotshade.txt`。

7. 影像輸出給報告

- 輸出兩張圖：
 - i. `text-spotshade_corrected.png`: 已做光照補償、spot shade 被去掉的灰階影像；
 - ii. `text-spotshade_binary.png`: 最後用來做 OCR 的二值影像。